

std::vector – operacje zaawansowane

Funkcje zaawansowane

Jeśli z góry znamy **rozmiar tablicy**, jaki będziemy potrzebowali, to możemy od razu zadeklarować vector o zdefiniowanym rozmiarze, przykładowo:

```
std::vector< int > vTab( 10 );
```

Wszystkie elementy takiego vectora będą miały wartość „neutralną”, tj.: dla liczb - zero, dla tekstów - brak tekstu, dla wskaźników -NULL, itd.. Oczywiście trzeba je wtedy później ustawić.

Jest to rozwiązanie lepsze (wydajniejsze) niż deklaracja vectora pustego z późniejszym dokładaniem elementów pojedynczo, bo pojedyncze dodawanie elementów z użyciem **push_back()**, przeladowuje pamięć co kilka wstawień, przez co aplikacja działa nieco wolniej.

Jeśli znamy **rozmiar docelowy**, w dodatku chcemy, aby wszystkie elementy były ustawione na **tą samą wartość**, to stosujemy deklarację jak poniżej. (vector będzie miał 10 elementów, każdy o wartości 33).

```
std::vector< int > vTab( 10, 33 );
```

Jeśli rozmiar vectora jest mały, a znamy **wszystkie początkowe wartości elementów**, to możemy je od razu ustawić podczas deklaracji. Uwaga! Używamy **nawiasów klamrowych**. (vector będzie miał 4 elementy, ich wartości jak ustawione).

```
vector< int > vTab { 11, 33, 44, 55 };
```

Zmiana rozmiaru vectora. W dowolnym momencie możemy zmienić rozmiar naszego vectora. Jeśli zwiększymy rozmiar, to wszystkie nowe elementy (dołożone z tyłu) będą miały ustawioną wartość neutralną. Jeśli zmniejszymy rozmiar, to wszystkie elementy, które się nie zmieszczą (odcięte z tyłu) zostaną utracone.

```
Vtab.resize( 100 );
```

Wstawienie elementu w dowolne miejsce vectora.

Można wstawiać elementy nie tylko na koniec, ale także w dowolne miejsce vectora. W takim wypadku tablica zostanie „rozsunięta”, a nowe elementy wstawione od miejsca, na które wskazuje iterator.

Ogólny zapis instrukcji:

vector.insert(lokalizacja(iterator), wartość);

lokalizacja(iterator) – lokalizacja elementu w vectorze, od którego wstawiamy nowe elementy. Jeśli w tej lokalizacji jest jakiś element, to będzie on przesunięty. Przykładowo, wstawiamy wartość **77** jako pierwszy element:

vTab.insert(vTab.begin(), 77);

vTab.begin() - iterator, wskazuje pozycję. Samo **.begin()** jest równoznaczne z pozycją o indeksie **[0]**.

Można też wstawić kilka elementów od razu.

Ogólny zapis instrukcji:

vector.insert(lokalizacja(iterator), ile_elementów, wartość);

Przykładowo, wstawiamy **4** elementy o wartości **88** od **3** elementu:

vTab.insert(vTab.begin() + 2, 4, 88);

vTab.begin() + 2 - iterator, wskazuje pozycję. **begin() + 2** jest równoznaczne z trzecim elementem, pozycją o indeksie **[2]**.

Można też usuwać elementy z dowolnego miejsca.

vTab.erase(lokalizacja(iterator));

Przykładowo, usuwamy element o indeksie 2.

vTab.erase(vTab.begin() + 2);

lub od razu usuwamy grupę elementów:

vTab.erase(lokalizacja_start(iterator), lokalizacja_do(iterator));

Przykładowo, usuwamy wszystkie elementy, poza pierwszy i ostatnim:

vTab.erase(vTab.begin() + 1, vTab.end() - 1);

vTab.end() - element za-ostatni. Miejsce w pamięci od którego już nie ma żadnego elementu vectora.